

PFL_TP1_G09_06

Representação Interna dos Polinómios

Estrutura

- Decidimos representar os polinómios como uma lista de tuplos, em que o primeiro elemento do tuplo é o coeficiente do monómio, e o segundo elemento é uma lista de tuplos (Char,Int), em que o Char é a variável e o Int é o expoente. Graças a isto, podemos representar polinómios do tipo $2x^2y^3+3z^3$, em que os monómios podem ter várias variáveis.
- Deste modo, o polinómio $2x^2y^3+3z^3$ ficará representado da seguinte forma :

```
[(2, [('x',2), ('y',3)]), (3, [('z',3)])].
```

Justificação

- Inicialmente, a ideia que nos ocorreu intuitivamente foi de representar os polinómios numa lista de tuplos, em que o primeiro elemento do tuplo seria o coeficiente e o segundo seria um tuplo com a variável e o expoente (ex: $2x^2 = (2, ('x',2))$). No entanto, deste modo não poderíamos representar um polinómio em que o monómio tivesse várias variáveis, pelo que a adaptação lógica foi tornar o 2º elemento do tuplo uma lista de tuplos.
- Achamos que este é o modo mais fácil e intuitivo de representar os polinómios, uma vez que olhando para a representação do polinómio, conseguimos perceber como é o polinómio facilmente, e assim apenas temos de trabalhar com listas e tuplos, que já têm funções pré-definidas para obtermos um certo elemento. Assim, seria fácil aceder quer aos elementos da lista, quer aos elementos do tuplo.

Funcionalidades

Normalizar um Polinómio na FR - normalizeString::[(Int, [(Char, Int)])]->String

- Esta função recebe um polinómio na forma representativa.
- Em primeiro lugar, começa por ordenar o polinómio segundo o critério que definimos (grau dos monómios)
- Depois, simplifica o polinómio, somando os elementos que podem ser somados dentro do polinómio, através da função simplificarPolinomio (procura elementos com o 2º elemento do tuplo iguais, somando depois os primeiros elementos e mudando num dos tuplos, removendo o outro tuplo).
- Por fim, remove os monómios com coeficiente 0, procurando os elementos em que o primeiro elemento do tuplo é 0, e removendo-os com uma função recursiva.
- É retornada uma string que representa o polinómio normalizado.

- Exemplo input: `normalizeString [(2, [('x',2), ('y',3)]), (3, [('z',4)])]`
- Exemplo output: `2*x^2*y^3+3*z^4`

Normalizar um Polinómio em String - `strNormalizeString::String->String`

- Semelhante em tudo á função acima, com a diferença que recebe o polinómio em string.
- Para passá-lo para a forma representativa, aplica a função `strToFr` do módulo `StringToFormat`, que aplica uma série de funções, transformando a string numa lista de tuplos.
- Depois, esta função funciona exatamente como a `normalizeString`.
- Exemplo input: `strNormalizeString "0*x^2 + 2*y + 5*z + y + 7*y^2"`
- Exemplo output: `7*y^2+3*y+5*z`

Soma de Polinómios na FR - `somaPolString::[(Int, [(Char, Int)])]->[(Int, [(Char, Int)])]->String`

- A função recebe 2 polinómios na sua forma representativa.
- Depois, concatenamos ambos os polinómios e normalizamos o polinómio resultante (uma vez que na normalização há soma de monómios que podem ser somados, cumprindo o nosso objetivo).
- É retornado o polinómio resultante da soma como uma string.
-
- Exemplo input: `somaPolString [(2, [('x',2), ('y',3)]), (3, [('z',4)])] [(1, [('x',2), ('y',4)]), (2, [('z',2)])]`
- Exemplo output: `x^2*y^4+2*x^2*y^3+3*z^4+2*z^2`

Soma de Polinómios em String - `strSomaPolString::String->String->String`

- A função recebe 2 polinómios como String.
- Depois, aplica a ambos a função de parsing `strToFr` do módulo `StringToFormat`, que transforma ambos os polinómios na sua forma representativa.
- Depois, a função comporta-se da mesma maneira que a função acima, retornando o polinómio resultado como String.
-
- Exemplo input: `strSomaPolString "x^2+2*y+5*z" "2*x^2+5*y+3"`
- Exemplo output: `3*x^2+7*y+5*z+3`

Multiplicação de Polinómios em FR - `multPNString::[(Int, [(Char, Int)])]->[(Int, [(Char, Int)])]->String`

- A função recebe 2 polinómios na sua FR.

- Nesta função, decidimos dividir a multiplicação em 3 partes : multiplicação de variáveis, multiplicação de monómios e multiplicação dos polinómios.
- Primeiro, multiplicamos as variáveis todas, percorrendo a lista recursivamente e somando os expoentes.
- Depois, multiplicamos os monómios todos, multiplicando o primeiro elemento de um com o segundo de outro, e aplicando a função de multiplicar as variáveis que falei acima ao 2º elemento do monómio resultante.
- Depois, aplicamos isto recursivamente a duas lista de monómios (=polinómio), obtendo um polinómio que é resultado da multiplicação de 2.
- O resultado é depois imprimido como string.
- Exemplo input: `multPNString [(2,[('x',2),('y',3)]),(3,[('z',4)])] [(1,[('x',2),('y',4)]),(2,[('z',2)])]`
- Exemplo output: `2*x^4*y^7+3*x^2*y^4*z^4+4*x^2*y^3*z^2+6*z^6`

Multiplicação de Polinómios em String - `strMultPNString::String->String->String`

- Comporta-se do mesmo modo que a função acima, com a exceção que recebe os polinómios em string, tendo de aplicar a função `strToFr` do `StringToFormat.hs` para os passar para a FR.
- Depois, faz exatamente o mesmo da função acima.
-
- Exemplo input: `strMultPNString "x^2+2*y+5*z" "2*x^2+5*y+3"`
- Exemplo output: `2*x^4+9*x^2*y+10*x^2*z+10*y^2+25*y*z+3*x^2+21`

Multiplicação de Polinómio em FR por Constante (EXTRA) - `multiplyByConstantString::[(Int, [(Char, Int)])->Int->String`

- A função recebe um polinómio na sua forma representativa e uma constante inteira.
- Depois aplica a função `multiplyByConstant`, em que basicamente multiplica o primeiro elemento de todos os monómios do polinómio pela constante e mantém o 2º elemento igual, isto tudo usando recursão.
- Depois, retorna o polinómio resultante como um String.
- Exemplo input: `multiplyByConstantString [(2,[('x',2),('y',3)]),(3,[('z',4)])] 4`
- Exemplo output: `8*x^2*y^3+12*z^4`

Multiplicação de Polinómio em String por Constante (EXTRA) - `strMultiplyByConstantString::String->Int->String`

- Comporta-se do mesmo modo que a função acima, com a exceção que recebe o polinómio em string, tendo de aplicar a função `strToFr` do `StringToFormat.hs` para passar o polinómio para a FR.

- Depois, faz exatamente o mesmo da função acima.
- Exemplo input: `strMultiplyByConstantString "x^2+2*y+5*z" 4`
- Exemplo output: `4*x^2+20*z+8*y`

Derivar um polinómio na FR - `derivePolynomialString::Char->[(Int, [(Char, Int)])]->String`

- No caso da derivação, decidimos pedir ao utilizador uma variável para derivar, tratando os monómios que não têm essa variável como constantes.
- A função recebe um char, que vai ser a variável a ser derivada, e um polinómio na forma representativa.
- Depois, é aplicada a função `derivePolynomial`, em que vamos derivar monómio a monómio através de recursão. Para derivar um monómio, temos ajuda da função `findV`, que procura a variável a derivar na lista com as variáveis de cada monómio (2º elemento do tuplo)

```
- se o 2º elemento não tiver a variável escolhida, fica (0, [('x',1)]) (tudo o que não tem a variável a derivar é tratado como constante)
- se o 2º elemento tiver a variável e for de tamanho 1 (só tem a variável), multiplicamos o 1º elemento pelo 2º elemento do único tuplo da lista (expoente), e reduzimos esse elemento em 1 unidade.
- se o 2º elemento tiver a variável e for de tamanho maior que 1, aplicamos o mesmo que acima, com a diferença que mantemos os outros elemento da lista de variáveis
```

- No fim, retorna-se o polinómio resultado como uma String.
- Exemplo input: `derivePolynomialString 'y' [(1, [('x',2), ('y',4)]), (2, [('z',2)])]`
- Exemplo output: `4*x^2*y^3`

Derivar um polinómio em string - `strDerivePolynomialString::Char->String->[(Int, [(Char, Int)])]->String`

- Comporta-se do mesmo modo que a função acima, com a exceção que recebe o polinómio em string, tendo de aplicar a função `strToFr` do `StringToFormat.hs` para passar o polinómio para a FR.
- Depois, faz exatamente o mesmo da função acima.
- Exemplo input: `strDerivePolynomialString 'y' "2*x^2+5*y+3"`
- Exemplo output: `"5"`

Derivar um polinómio (segunda derivada) em FR (EXTRA) - `secondDerivePolynomialString::Char->[(Int, [(Char, Int)])]->String`

- Comportamento igual à função descrita acima, com a diferença que cada elemento é derivado 2 vezes, aplicando-se a função `deriveElement` 2 vezes a cada elemento.
- Fizemos isto usando recursão.

- No fim, retorna-se o polinómio resultado como uma String.
- Exemplo input: `secondDerivePolynomialString 'x' [(2,[('x',2)]),(5,[('y',1)]),(3,[('x',0)])]`
- Exemplo output: `"4"`

Derivar um polinómio (segunda derivada) em String (EXTRA) -

`strSecondDerivePolynomialString::Char->String->String`

- Comporta-se da mesma maneira que a função acima, só que recebe o polinómio em string, aplicando-lhe a função de parse `strToFr` presente no `StringToFormat` para passar o polinómio para a FR.
- Depois, faz exatamente o mesmo que a função acima, retornando o resultado como uma string.
- Exemplo input: `strSecondDerivePolynomialString 'x' "2*x^2+5*y+3"`
- Exemplo output: `"4"`

Grau de um polinómio (EXTRA) - `polynomialDegree::[(Int, [(Char, Int)])]->Int`

- Esta função recebe um polinómio na forma representativa como argumento.
- Depois, soma os expoente do primeiro elemento do polinómio normalizado (ao ser normalizado, o 1º elemento da lista será o monómio de maior grau).
- Para somar os expoentes, usamos recursão sobre o 1º elemento do 2º elemento do 1º tuplo da lista, em que vamos somando todos os 2ºs elementos da lista de variáveis (que são os expoentes).
- No fim, retornamos o resultado como um inteiro.
- Exemplo input: `polynomialDegree [(1,[('x',2),('y',4)]),(2,[('z',2)])]`
- Exemplo output: `6`

Menu (EXTRA)

- Decidimos também criar um menu para promover ao utilizador uma experiência mais cómoda a utilizar o nosso programa. Nesse menu, os polinómios são todos dados como string pelo utilizador, porque é a maneira mais simples e lógica de ele os colocar, ao invés da FR que não é assim tão fácil para o utilizador usar, principalmente se colocar polinómios muito complexos.
- Para utilizar o menu, dar `:l Menu.hs` para carregar o módulo do menu, e depois escrever no "ghci" `main`, para invocar o método `main`, que chama o nosso menu.